

Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 2



Nama:

Habiburrahman Ikwan Mujahidin

NIM:

20240801149

UNIVERSITASA ESA UNGGUL

TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

Dalam pengembangan aplikasi mobile menggunakan Flutter, penyusunan tampilan interface, pengelolaan daftar data, dan perpindahan antarhalaman merupakan bagian penting yang perlu dipahami. Flutter menyediakan berbagai widget seperti Container, Row, Column, Expanded, ListView.builder, Card, dan ListTile untuk membangun antarmuka yang terstruktur. Selain itu, Navigator dapat digunakan untuk melakukan perpindahan halaman serta mengirimkan data dari satu halaman ke halaman lainnya.

2. Tujuan Praktikum

- Memahami penggunaan Container, Padding, Row, Column, dan Expanded.
- Menampilkan data menggunakan ListView.builder.
- Membuat model data sederhana menggunakan class Dart.
- Membuat navigasi antarhalaman menggunakan Navigator.push() dan Navigator.pop().

3. Teori Dasar

- Container : Kotak serbaguna: ukuran, warna, border, radius, padding, margin
- Padding : Memberi jarak di sekeliling widget anak
- Row / Column : Menyusun anak secara horizontal / vertikal
- Expanded : Membuat anak mengisi sisa ruang pada Row/Column
- ListView.builder : Daftar yang di-render sesuai kebutuhan (efisien untuk data banyak)
- Card + ListTile : Kartu berisi baris standar: ikon, judul, subjudul, ikon akhir
- Navigator Mengelola tumpukan (stack) halaman: push menambah, pop menutup

4. Langkah Praktikum

Pertama kita buka terminal dan buat project baru untuk praktikum ini

```
flutter create praktikum1
```

```
cd praktikum1
```

```
flutter run
```

5. Implementasi Layout kartu profil

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
```

```

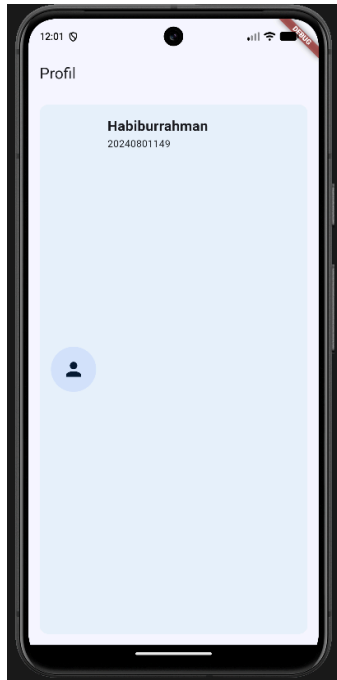
        return MaterialApp(
          @override
Widget build(BuildContext context) {
  return MaterialApp(
    title: 'Praktikum 2',
    theme: ThemeData(
      colorSchemeSeed: Colors.blue,
      useMaterial3: true,
    ),
    home: const ProfilePage(),
  );
}
}

class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Profil'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
      child: Container(
        padding: const EdgeInsets.all(16),
        decoration: BoxDecoration(
          color: Colors.blue.shade50,
          borderRadius: BorderRadius.circular(12),
        ),
        child: Row(
          children: [
            const CircleAvatar(
              radius: 32,
              child: Icon(
                Icons.person,
                size: 32,
              ),
            ),
            const SizedBox(width: 16),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: const [
                  Text(
                    'Habiburrahman',
                    style: TextStyle(
                      fontSize: 20,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                  Text('20240801149'),
                ],
              ),
            ),
          ],
        ),
      ),
    ),
  );
}
}

```

Singkatnya, `main()` menjalankan `MyApp`, sebuah `StatelessWidget` yang membangun `MaterialApp` dan menampilkan halaman `ProfilePage`. `ProfilePage` juga `StatelessWidget` yang menyusun `Scaffold` dengan `AppBar` berjudul "Profil". Di dalam body, terdapat `Padding` yang membungkus `Container` berwarna biru muda dengan sudut melengkung 12. Di dalamnya ada `Row` berisi: `CircleAvatar` (icon), lalu `Expanded > Column` yang menampilkan nama saya "Habiburrahman Ikwan M" dan NIM "20240801149" secara vertikal rata kiri.



6. Implementasi model data dan listview

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Praktikum 2',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const MenuPage(),
    );
  }
}
```

```

}

class Makanan {
  final String nama;
  final int harga;

  const Makanan(this.nama, this.harga);
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];

class MenuPage extends StatelessWidget {
  const MenuPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Menu'),
      ),
      body: ListView.builder(
        itemCount: daftarMenu.length,
        itemBuilder: (context, index) {
          final item = daftarMenu[index];

          return Card(
            margin: const EdgeInsets.symmetric(
              horizontal: 12,
              vertical: 6,
            ),
            child: ListTile(
              leading: const Icon(Icons.restaurant),
              title: Text(item.nama),
              subtitle: Text('Rp ${item.harga}'),
              trailing: const Icon(Icons.chevron_right),
            ),
          );
        },
      ),
    );
  }
}

```

Kode ini membuat aplikasi daftar menu sederhana. `main()` menjalankan `MyApp`, yang menampilkan `MaterialApp` dengan halaman utama `MenuPage`. Data menu disimpan di `daftarMenu` berisi nama dan harga. Di `MenuPage`, `ListView.builder` menampilkan setiap menu dalam bentuk `Card` dan `ListTile`, lengkap dengan ikon, nama, harga, dan tanda panah. Hasilnya, layar menampilkan daftar menu makanan yang bisa digulir. Karena datanya tetap, halaman ini memakai `StatelessWidget`.

7. Implementasi Navigasi ke halaman Detail

```

import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class Makanan {
  final String nama;
  final int harga;

  const Makanan(this.nama, this.harga);
}

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
];

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Praktikum 2',
      theme: ThemeData(
        colorSchemeSeed: Colors.blue,
        useMaterial3: true,
      ),
      home: const MenuPage(),
    );
  }
}

class MenuPage extends StatelessWidget {
  const MenuPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Daftar Menu'),
      ),
      body: ListView.builder(
        itemCount: daftarMenu.length,
        itemBuilder: (context, index) {
          final item = daftarMenu[index];

          return Card(
            margin: const EdgeInsets.symmetric(
              horizontal: 12,
              vertical: 6,
            ),
            child: ListTile(
              leading: const Icon(Icons.restaurant),
              title: Text(item.nama),
              subtitle: Text('Rp ${item.harga}'),
            ),
          );
        },
      ),
    );
  }
}

```

```

        trailing: const Icon(Icons.chevron_right),
        onTap: () {
          Navigator.push(
            context,
            MaterialPageRoute(
              builder: (_) => DetailPage(
                makanan: item,
              ),
            ),
          );
        },
      ),
    ),
  );
},
),
);
}
}
}

class DetailPage extends StatelessWidget {
  final Makanan makanan;

  const DetailPage({
    super.key,
    required this.makanan,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(makanan.nama),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.restaurant_menu,
              size: 80,
            ),
            const SizedBox(height: 16),
            Text(
              makanan.nama,
              style: const TextStyle(
                fontSize: 24,
              ),
            ),
            const SizedBox(height: 24),
            Text('Rp ${makanan.harga}'),
            const SizedBox(height: 24),
            ElevatedButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Kembali'),
            ),
          ],
        ),
      ),
    ),
  ),
);

```

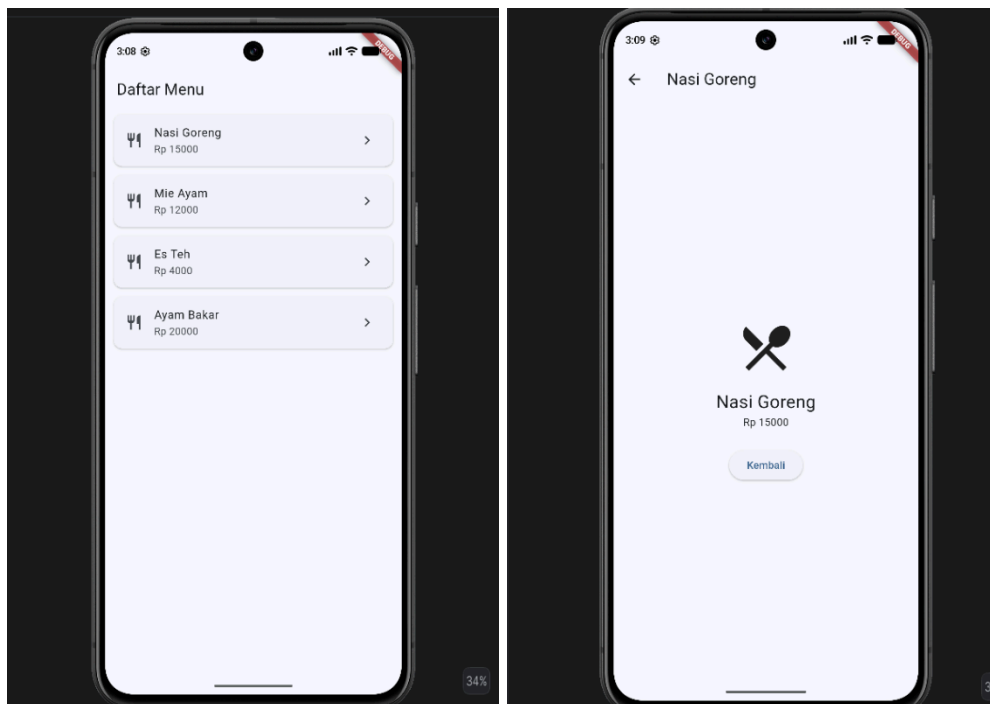
```

    );
  }
}

```

Kode ini membuat aplikasi daftar menu yang bisa diklik untuk melihat detail. main() menjalankan MyApp, yang menampilkan MenuPage berisi daftar menu dari daftarMenu. Setiap menu ditampilkan dalam Card dan ListTile; jika diklik, aplikasi berpindah ke DetailPage sambil membawa data Makanan yang dipilih. Di DetailPage, layar menampilkan ikon, nama, harga, dan tombol Kembali yang berfungsi untuk kembali ke daftar menu. Karena semua data tetap, kedua halaman ini memakai StatelessWidget.

Hasil akhir:



8. Latihan Mandiri

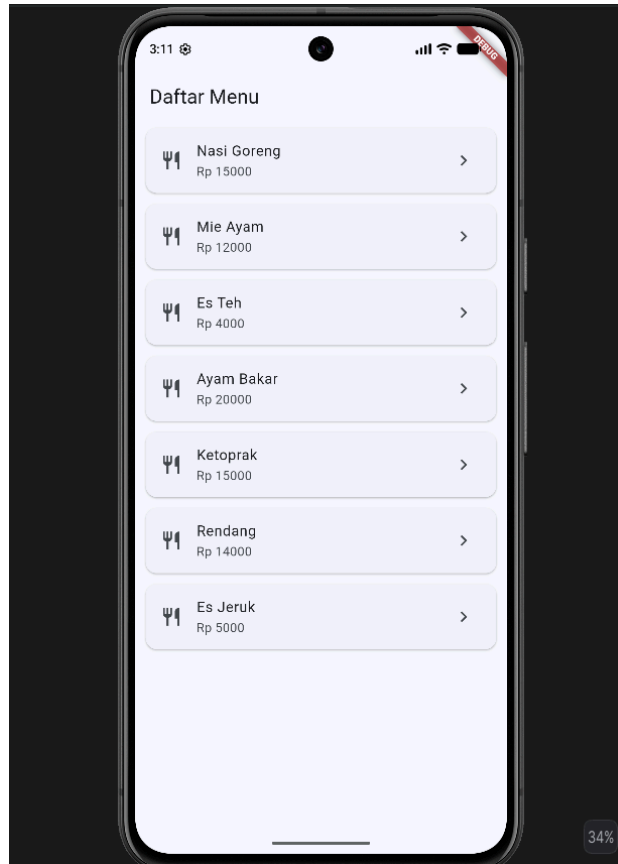
- Tambahkan 3 menu baru ke daftarMenu dan pastikan daftar tetap dapat di-scroll

```

const daftarMenu = [
  Makanan('Nasi Goreng', 15000),
  Makanan('Mie Ayam', 12000),
  Makanan('Es Teh', 4000),
  Makanan('Ayam Bakar', 20000),
  Makanan('Ketoprak', 15000),
  Makanan('Rendang', 14000),
  Makanan('Es Jeruk', 5000),
];

```

Disini saya hanya menambahkan jumlah daftar makanan dari yang sebelumnya 4 menjadi 7, code sisanya sama seperti diatas. Hasil akhir:



- Tambahkan properti deskripsi pada class Makanan dan tampilkan di DetailPage. Sekarang alurnya jadi lebih lengkap. Data Makanan tidak hanya menyimpan nama dan harga, tapi juga deskripsi. Saat aplikasi dijalankan, main() memanggil MyApp yang menampilkan MenuPage berisi daftarMenu (7 item, masing-masing dengan nama, harga, dan deskripsi). Ketika salah satu menu ditekan, Navigator.push membuka DetailPage sambil membawa seluruh objek Makanan yang dipilih. Di DetailPage, data tersebut ditampilkan lengkap: ikon, nama, harga, **dan deskripsi** di bawahnya.

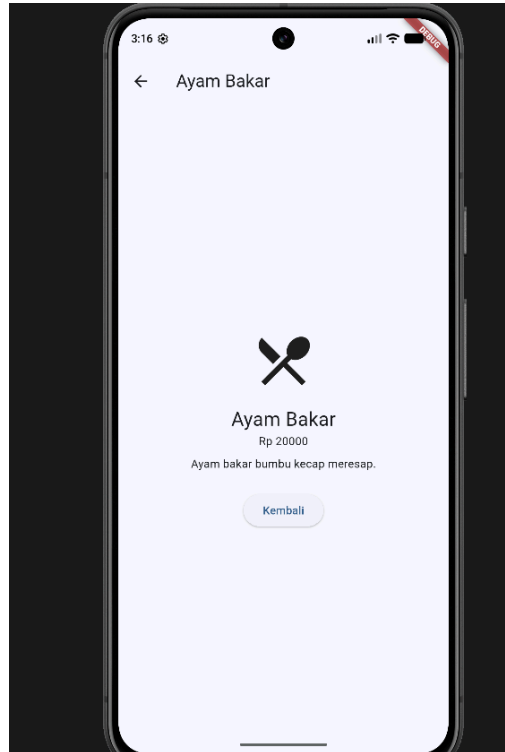
```
class Makanan {
    final String nama;
    final int harga;
    final String deskripsi;

    const Makanan(
        this.nama,
        this.harga,
        this.deskripsi,
    );
}
```

```
const daftarMenu = [
```

```
Makanan (
  'Nasi Goreng',
  15000,
  'Nasi goreng dengan cita rasa gurih menakjubkan.',
),
```

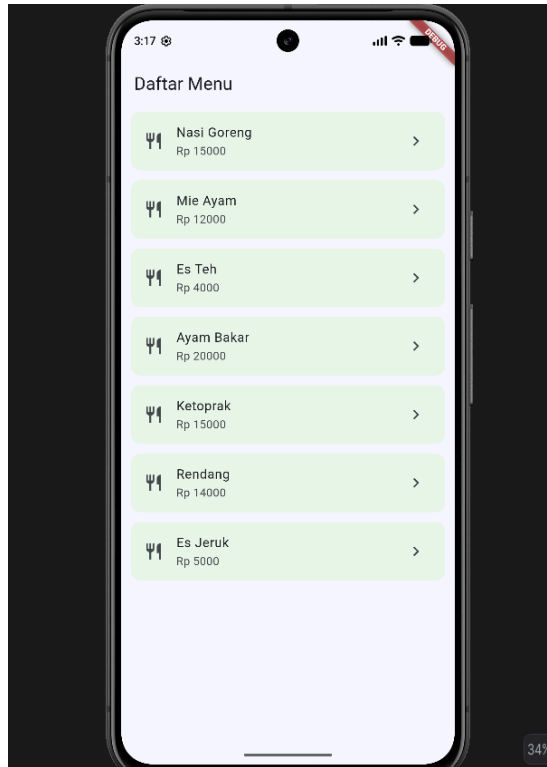
Hasil akhirnya:



- Ganti Card dengan Container yang memiliki warna latar dan sudut membulat.

```
return Container(
  margin: const EdgeInsets.symmetric(
    horizontal: 12,
    vertical: 6,
  ),
  decoration: BoxDecoration(
    color: Colors.green.shade50,
    borderRadius: BorderRadius.circular(12),
  ),
  child: ListTile(
```

Disini saya menambahkan container dengan latar warna hijau shade 50 dan borderRadius circular agar sudut membulat. Hasil akhir:

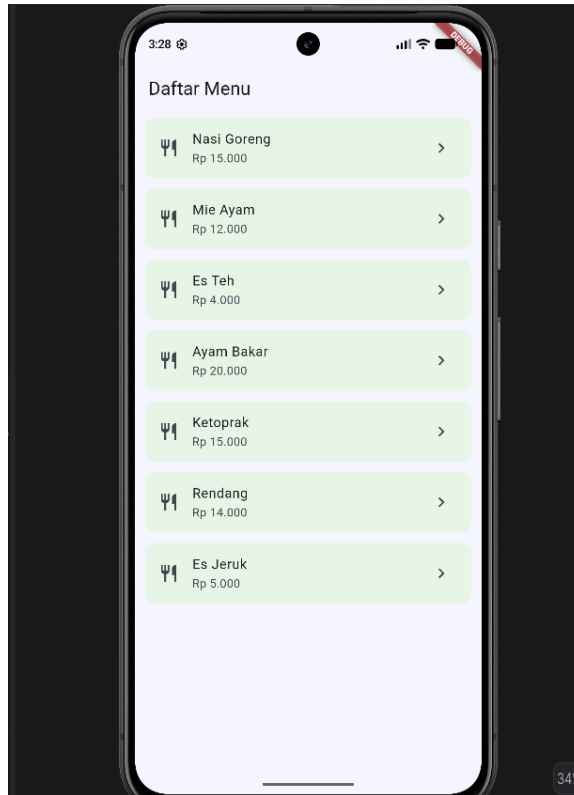


- Tampilkan harga dengan format ribuan (misalnya 15.000).

```
String formatRibuan(int angka) {  
    String a = angka.toString();  
    String hasil = '';  
    int count = 0;  
  
    for (int i = a.length - 1; i >= 0; i--) {  
        hasil = a[i] + hasil;  
        count++;  
        if (count % 3 == 0 && i != 0) {  
            hasil = '.' + hasil;  
        }  
    }  
    return hasil;  
}
```

Cara kerjanya dimulai dengan mengubah angka menjadi teks lewat `angka.toString()`, lalu menyiapkan wadah kosong bernama `hasil` dan penghitung `count` yang mulai dari nol. Setelah itu, fungsi membaca teks angka tersebut dari belakang ke depan menggunakan perulangan `for`. Setiap digit yang dibaca ditambahkan ke depan `hasil`, lalu `count` ditambah satu. Nah, setiap kali `count`

sudah mencapai kelipatan 3 (artinya sudah terkumpul 3 digit), fungsi akan menyisipkan titik . di depan hasil tapi hanya jika digit yang sedang diproses bukan digit paling awal ($i \neq 0$), supaya tidak ada titik nyasar di depan angka. Sebagai contoh, angka 15000 dibaca mundur menjadi 0, 0, 0, 5, 1; setelah tiga digit pertama terkumpul, titik disisipkan sehingga hasil menjadi 000, lalu bertambah jadi 5.000, dan akhirnya 15.000. Hasil akhir:



9. Tugas aplikasi daftar kontak dengan ketentuan:

Minimal 6 kontak (nama, nomor telepon, email) yang disimpan dalam list berisi objek class.

- Halaman utama menampilkan daftar kontak dengan ListView.builder dan ListTile (avatar berisi huruf pertama nama).
- Mengetuk kontak membuka halaman detail yang menampilkan seluruh data kontak dan tombol kembali.

10. Implementasi tugas

```

class Kontak {
  final String nama;
  final String telepon;
  final String email;
  const Kontak(this.nama, this.telepon, this.email);
}

```

Data kontak disimpan di daftarKontak, berisi enam objek Kontak yang masing-masing memiliki nama, telepon, dan email.

```

return ListTile(
  leading: CircleAvatar(
    child: Text(kontak.nama[0]),
  ),
  title: Text(kontak.nama),
  subtitle: Text(kontak.telepon),
  trailing: const Icon(Icons.chevron_right),

```

Di KontakPage, ListView.builder menampilkan setiap kontak dalam bentuk ListTile. Bagian kiri (leading) memakai CircleAvatar berisi huruf pertama nama, judulnya nama, subjudulnya telepon, dan di kanan ada ikon panah.

```

onTap: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => DetailKontakPage(kontak: kontak),
    ),
  );
},

```

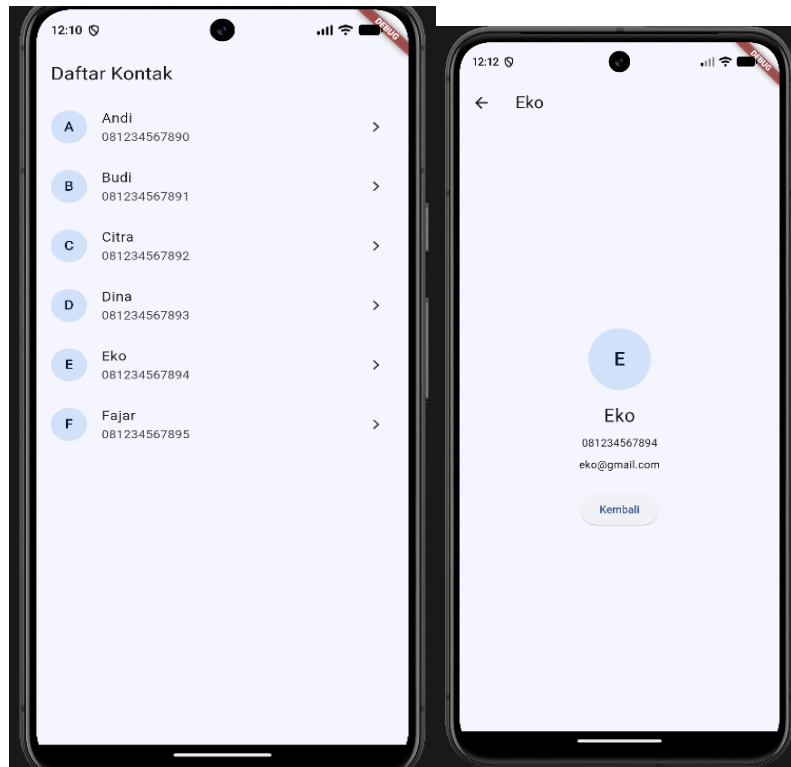
Ketika salah satu kontak ditekan, onTap menjalankan Navigator.push untuk membuka DetailKontakPage sambil membawa objek Kontak yang dipilih.

```

CircleAvatar(
  radius: 40,
  child: Text(kontak.nama[0], style: const TextStyle(fontSize: 24)),
),
const SizedBox(height: 16),
Text(kontak.nama, style: const TextStyle(fontSize: 24)),
const SizedBox(height: 8),
Text(kontak.telepon),
const SizedBox(height: 8),
Text(kontak.email),

```

Di DetailKontakPage, layar menampilkan avatar besar berisi huruf awal nama, lalu nama, telepon, dan email kontak tersebut. Hasil akhir:



11. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Flutter menyediakan berbagai widget yang dapat digunakan untuk membuat tampilan aplikasi secara terstruktur. Pada praktikum ini, class Makanan digunakan untuk memodelkan data makanan yang terdiri dari nama dan harga, kemudian data tersebut ditampilkan menggunakan ListView.builder, Card, dan ListTile.

Selain itu, penggunaan Navigator.push() memungkinkan pengguna berpindah dari halaman daftar menu ke halaman detail dengan membawa data makanan yang dipilih. Sementara itu, Navigator.pop() digunakan untuk kembali ke halaman sebelumnya.

12. Pertanyaan Refleksi

- Apa perbedaan ListView biasa dengan ListView.builder?
: ListView biasa membuat semua item sekaligus di awal, jadi cocok untuk daftar yang jumlahnya sedikit dan tetap. Sedangkan ListView.builder membuat item satu per satu sesuai kebutuhan, hanya item yang terlihat di layar yang dibangun, dan sisanya dibangun saat pengguna menggulir.
- Mengapa Row yang berisi teks panjang dapat menyebabkan overflow, dan bagaimana Expanded membantu?

: Row menempatkan anak-anaknya secara horizontal tanpa batas lebar otomatis. Kalau salah satu Text yang panjangnya melebihi lebar layar, teks itu akan keluar dari area yang tersedia, sehingga muncul error strip . Expanded membantu dengan cara mengambil sisa ruang yang tersedia dan membatasi lebar anak di dalamnya, sehingga Text otomatis dibungkus ke baris berikutnya alih-alih keluar dari layar.

- Bagaimana data dikirim dari halaman daftar ke halaman detail pada praktikum ini?

: Data dikirim melalui parameter konstruktor saat navigasi. Di halaman daftar, ketika item ditekan, Navigator.push dipanggil dan di dalam MaterialPageRoute dibuat instance halaman detail sambil mengirim objeknya.